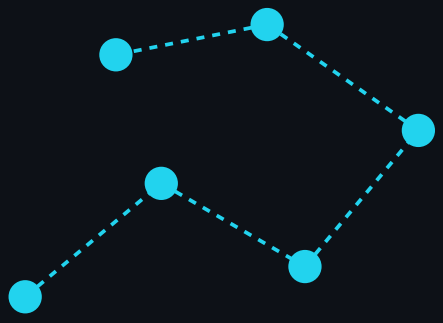


# Gephyra

## Graphical interface - user manual

Water-bridge network discovery, clustering and visualization

Version 0.3.0 - October 2026

Peralta-Morales, M. F., Sciortino, G. & Marechal, J.-D.

https://github.com/insilichem/Gephyra

# Contents

1. What the interface does 3

2. Installation and start-up 3

3. The window at a glance 5

4. Step 1 - Pathway Analysis 7

5. Step 2 - Clustering 9

6. Step 3 - Visualization 11

7. Output files 14

8. Troubleshooting 14

9. Command-line equivalents 14

10. Video walkthrough 15

# 1. What the interface does

Gephyra finds chains of water molecules that connect a chosen *root* atom (for example a ligand oxygen) to the surrounding solvent in a molecular dynamics trajectory, groups the chains that recur into spatial channels, and writes ready-to-run scenes for molecular viewers.

The graphical interface runs the whole workflow in three steps, without memorising command-line options:

- **1. Pathway Analysis** reads the topology and trajectory and writes every water bridge, frame by frame, to a results file (`.jsonl`).
- **2. Clustering** groups the bridges that recur in space into channels and writes a cluster file (`.json`).
- **3. Visualization** writes a script for UCSF ChimeraX, UCSF Chimera, PyMOL or VMD that loads the protein, the ligand and the water network and shows them together.

Every step shows its progress live, can be cancelled, and ends with a confirmation that lists the files written and the folder they are in.

# 2. Installation and start-up

## Requirements

- **Python 3.10 or newer; 3.11 recommended**, in a dedicated conda environment (below). Using a separate environment keeps the Python version and libraries Gephyra needs from clashing with other software, such as a PyMOL installation that ships its own Python.
- Conda (Miniconda, Miniforge or Anaconda) to create that environment.
- MDAnalysis, NetworkX, NumPy and SciPy, installed together with Gephyra.
- At least one molecular viewer to look at the results: UCSF ChimeraX, UCSF Chimera, PyMOL or VMD.

## Installing Gephyra

In a terminal, download Gephyra, create its environment and install it:

```
git clone https://github.com/insilichem/Gephyra.git
cd Gephyra
conda create -n gephyra -c conda-forge python=3.11 "tk=*=xft*"
conda activate gephyra
pip install .
```

The same environment, with the scientific libraries also taken from conda-forge, can be created from the file shipped with Gephyra:

```
git clone https://github.com/insilichem/Gephyra.git
cd Gephyra
conda env create -f environment.yml
conda activate gephyra
pip install .
```

`"tk=*=xft*"` selects the build of the Tk graphics library with smooth, anti-aliased fonts; with the default build the interface text looks blocky. Check the installation with `gephyra --version`.

**Tip.** Run `conda activate gephyra` in every new terminal before using Gephyra. If the `gephyra` command is not found, the environment is not active.

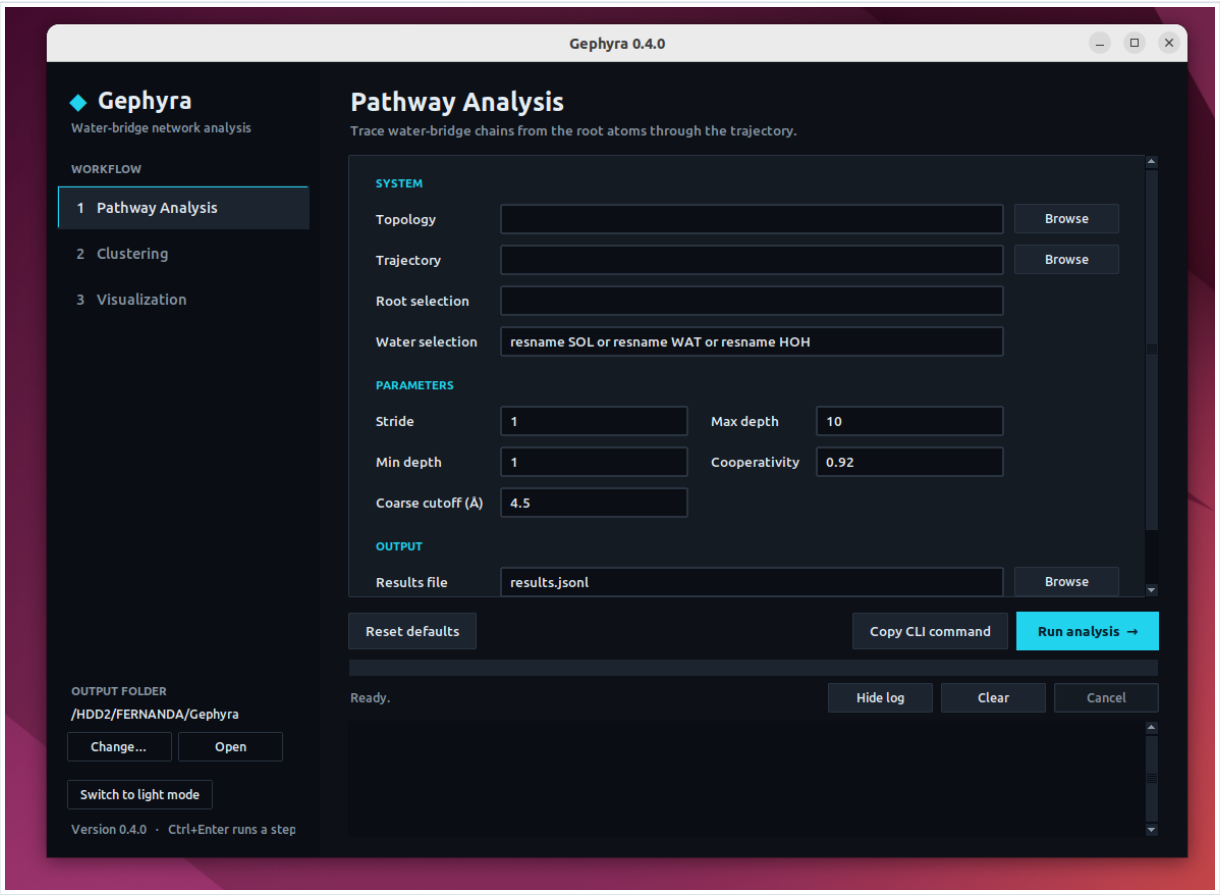
## Starting the interface

```
conda activate gephyra
gephyra gui
# or
gephyra-gui
```

The window opens on step 1. The folder you start it from is the default output folder (you can change it at any time, see section 3).

**Tip.** If the text looks blocky or pixelated, Gephyra is running in a Python whose Tk library has no anti-aliased fonts, and the interface says so in its log. This does not happen in the environment above; in another conda environment, install the right build with `conda install -c conda-forge "tk=*=xft*"` and restart the interface.

# 3. The window at a glance

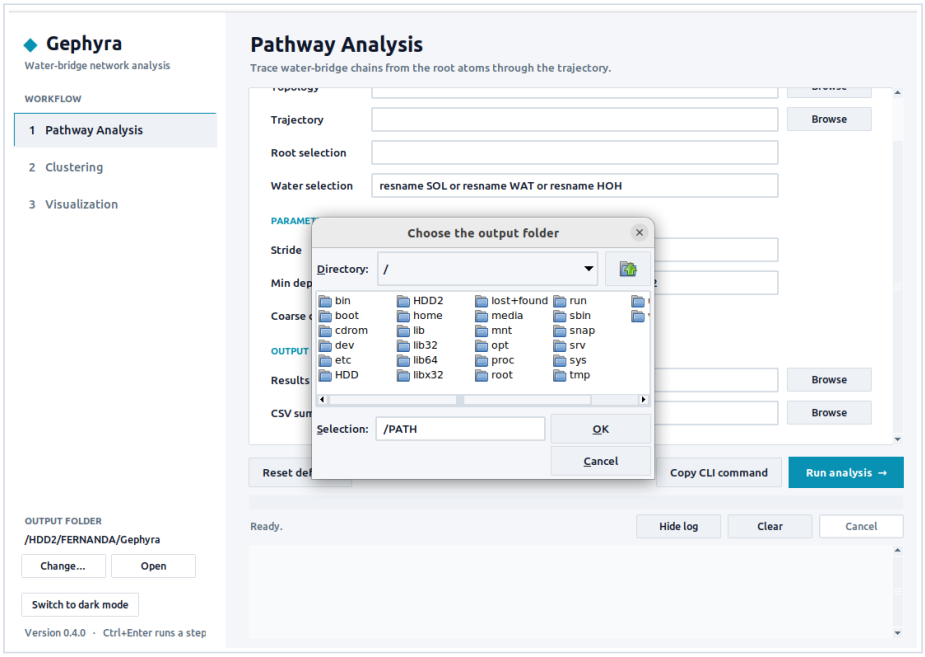



*The main window: sidebar with the three steps and the output folder, the form, the progress bar and the log.*

| Area | What it does |
|---|---|
| **Sidebar: workflow** | The three steps. Click one to open its form; the current step is highlighted. |
| **Sidebar: output folder** | Where all results are saved. **Change...** picks another folder, **Open** shows it in your file manager. |
| **Sidebar: theme** | Switches between the dark and the light theme. The choice is remembered. |
| **Form** | The options of the current step, grouped in sections. Hover over a field to see a short explanation. **Browse** opens a file picker. |
| **Buttons** | **Reset defaults** restores the standard values, **Copy CLI command** copies the equivalent command line, and the highlighted button runs the step (also Ctrl+Enter). |
| **Progress bar and status** | Show what is running and how far it has got. **Cancel** stops a running step after the current frame. |
| **Log** | Messages from Gephyra as they happen: warnings in amber, errors in red, saved files in green. **Hide log** and **Clear** manage it. |

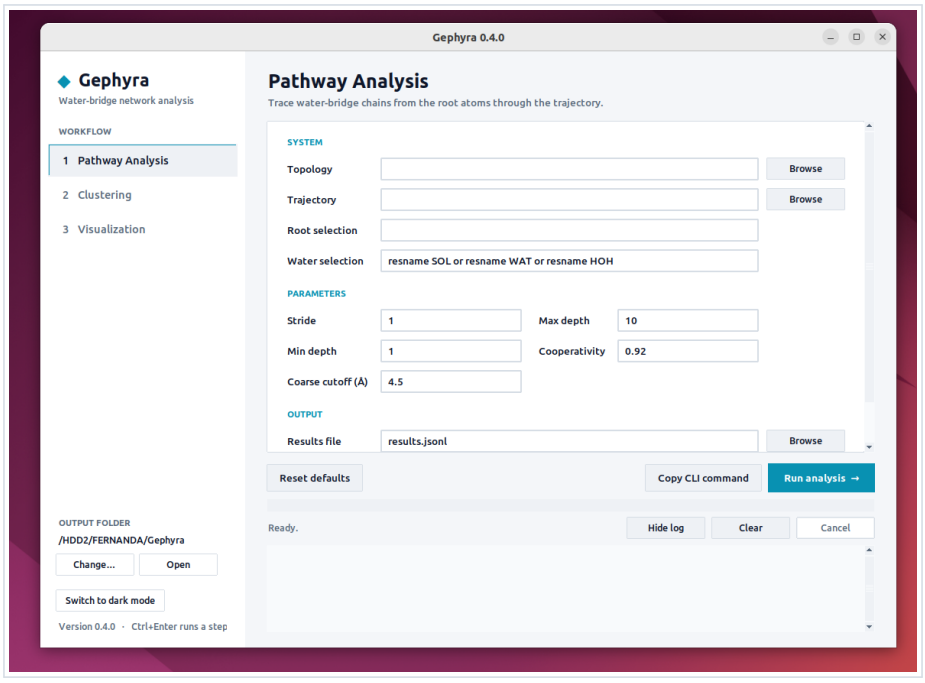
## Where results are saved

Output fields such as *Results file* or *Script name* hold file **names**, which are saved inside the output folder shown in the sidebar. Type an absolute path in a field to save that file somewhere else. If the output folder does not exist, the interface offers to create it.




*Choosing the output folder with Change... in the sidebar.*

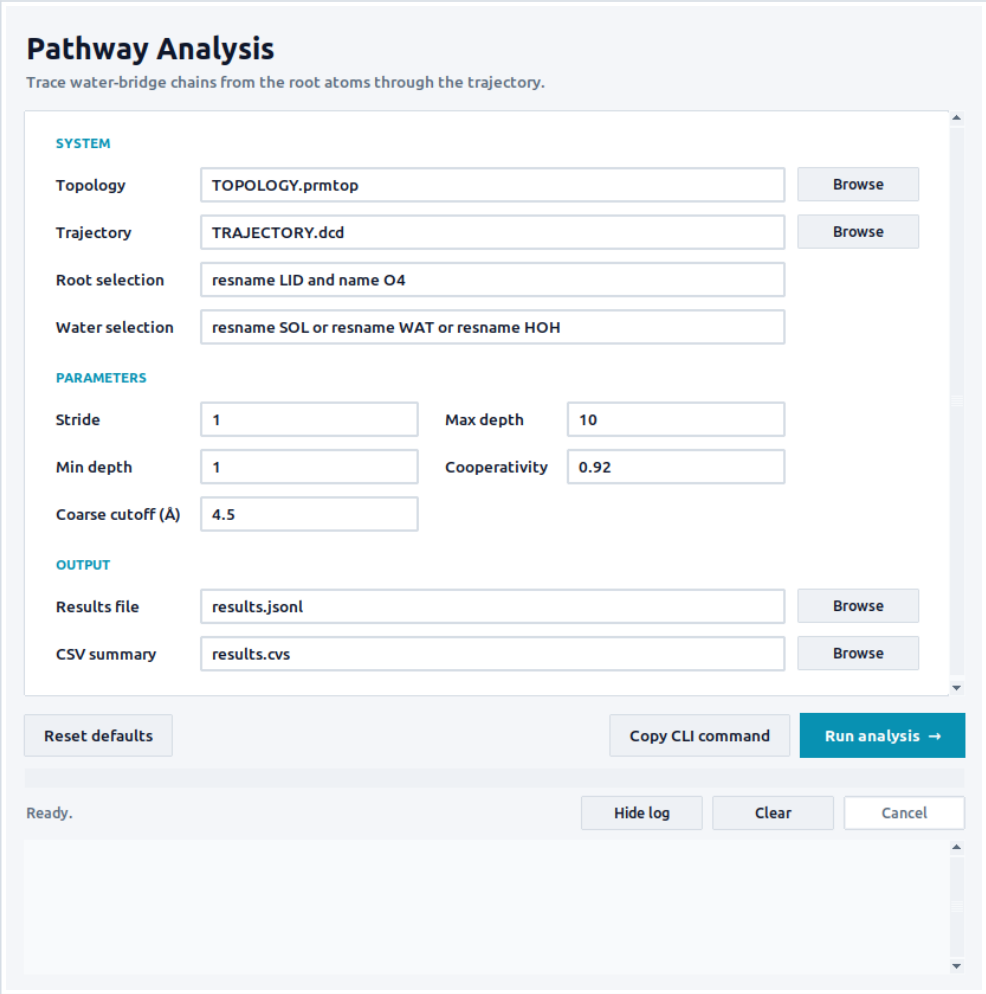
After a step finishes, its confirmation window lists every saved file with its folder, and offers **Open folder** and **Save a copy to...**, so results can also be stored in another location after the calculation.




*The same window in the light theme.*

# 4. Step 1 - Pathway Analysis

Finds the water bridges in every analysed frame of the trajectory.

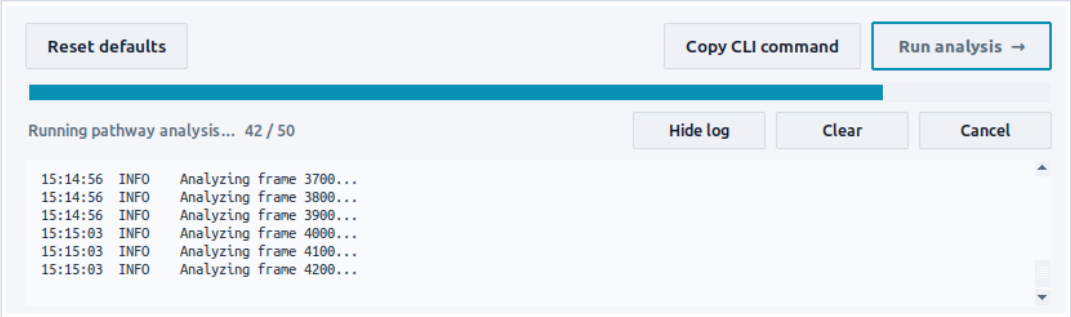



*Step 1, Pathway Analysis, filled in with a topology, a trajectory and a root selection.*

| Field | Meaning | Default |
|---|---|---|
| **Topology** | Structure/topology file of the system (.prmtop, .psf, .tpr, .pdb, ...). Bond information enables the full hydrogen-bond geometry criterion. | required |
| **Trajectory** | The frames to analyse (.dcd, .xtc, .nc, ...). For a single structure, choose a coordinate file such as a .pdb. | required |
| **Root selection** | MDAnalysis selection of the atom(s) the bridges start from, e.g. `resname LIG and name O4`. | required |
| **Water selection** | MDAnalysis selection of the bridging solvent. | SOL, WAT or HOH |
| **Stride** | Analyse every N-th frame. Larger values are faster. | 1 |
| **Max depth / Min depth** | Longest and shortest chains (number of waters) that are kept. | 10 / 1 |
| **Cooperativity** | Per-step discount along a chain, between 0 and 1 (1 switches it off). | 0.92 |
| **Coarse cutoff** | Neighbour search radius in angstrom. | 4.5 |
| **Results file** | Name of the frame-by-frame results file in the output folder. | results.jsonl |

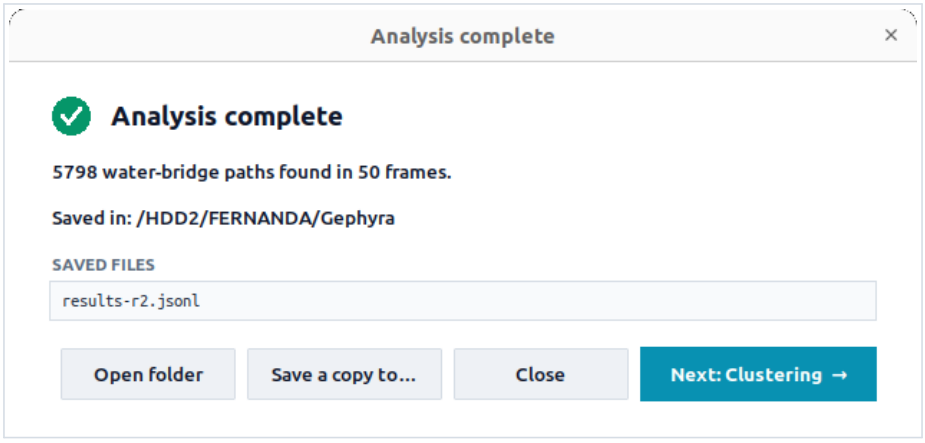
| Field | Meaning | Default |
|---|---|---|
| **CSV summary** | Optional table of all paths, for spreadsheets. | - |

**Tip.** For a first look at a long trajectory, start with a stride of 10-50. The analysis time grows with the number of frames and with Max depth.




*While the analysis runs: the progress bar counts frames and the log streams messages.*

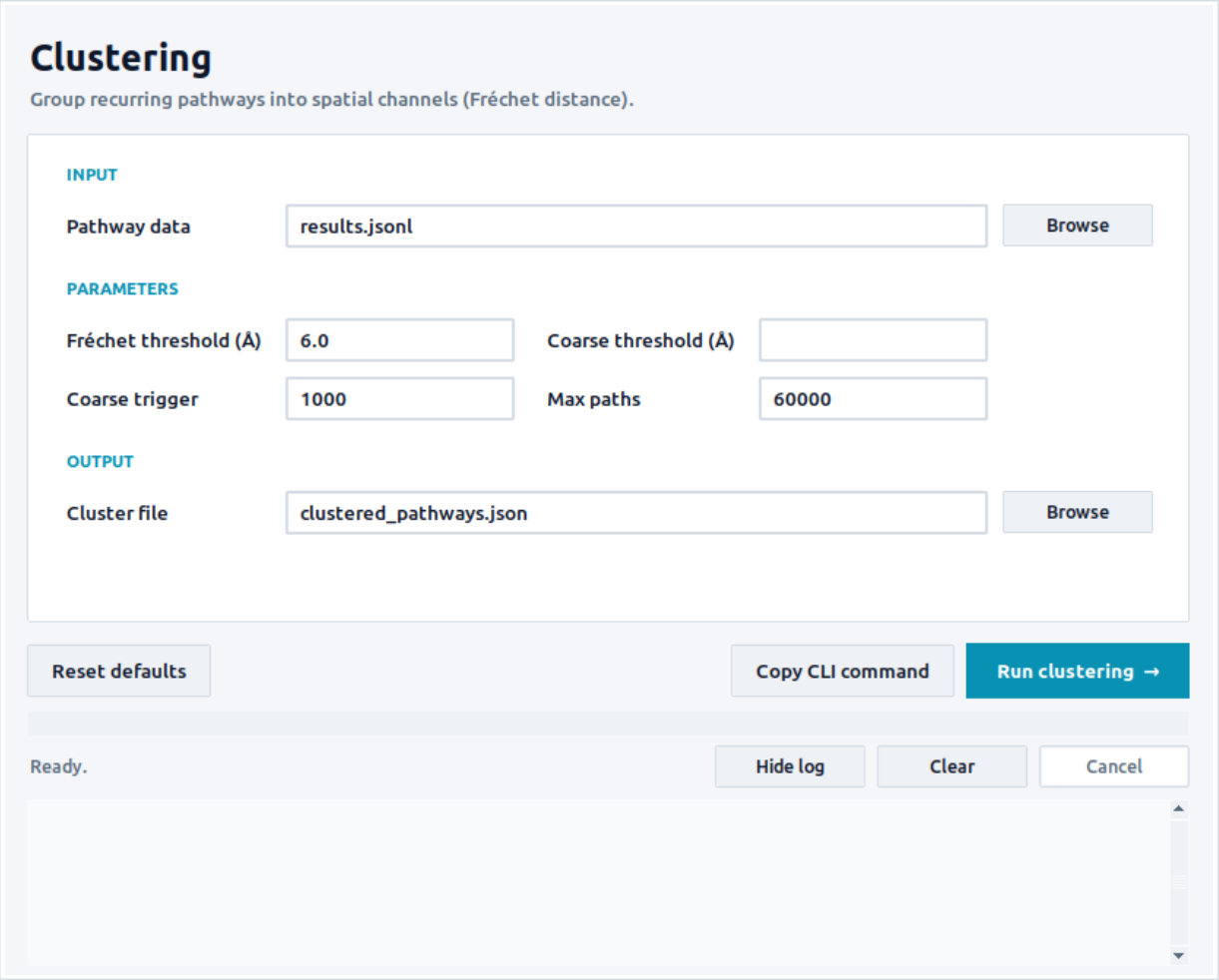
Click **Run analysis**. The progress bar counts the frames; when it finishes, a confirmation reports how many water-bridge paths were found and where the files were saved. Click **Next: Clustering** to continue: the results file is already filled in on the next step.




*The confirmation shown when the analysis finishes, listing the saved files and their folder.*

# 5. Step 2 - Clustering

Groups the paths found in step 1 into recurring spatial channels. Different water molecules occupying the same channel at different times end up in the same cluster.



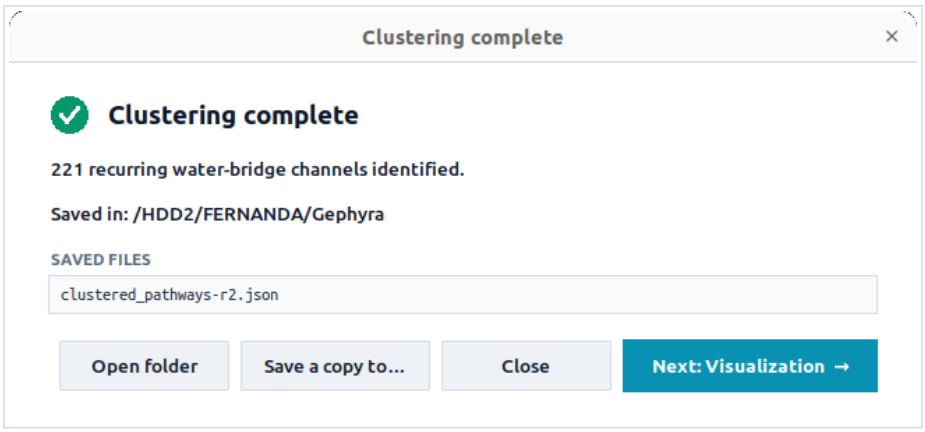

*Step 2, Clustering, with the pathway data filled in from step 1.*

| Field | Meaning | Default |
|---|---|---|
| **Pathway data** | The results file of step 1 (filled in automatically). | - |
| **Frechet threshold** | How similar two paths must be (in angstrom) to belong to the same cluster. Lower values give more, finer clusters; 3-8 is a typical range. | 6.0 |
| **Coarse threshold / trigger** | Settings of a fast pre-screening used for very large data sets; the defaults suit most cases. | auto / 1000 |
| **Max paths** | Memory cap: beyond this, only the most frequent paths are clustered. | 60000 |
| **Cluster file** | Name of the cluster file in the output folder. | clustered_pathways.json |




*While clustering runs: progress through the Frechet distance matrix.*

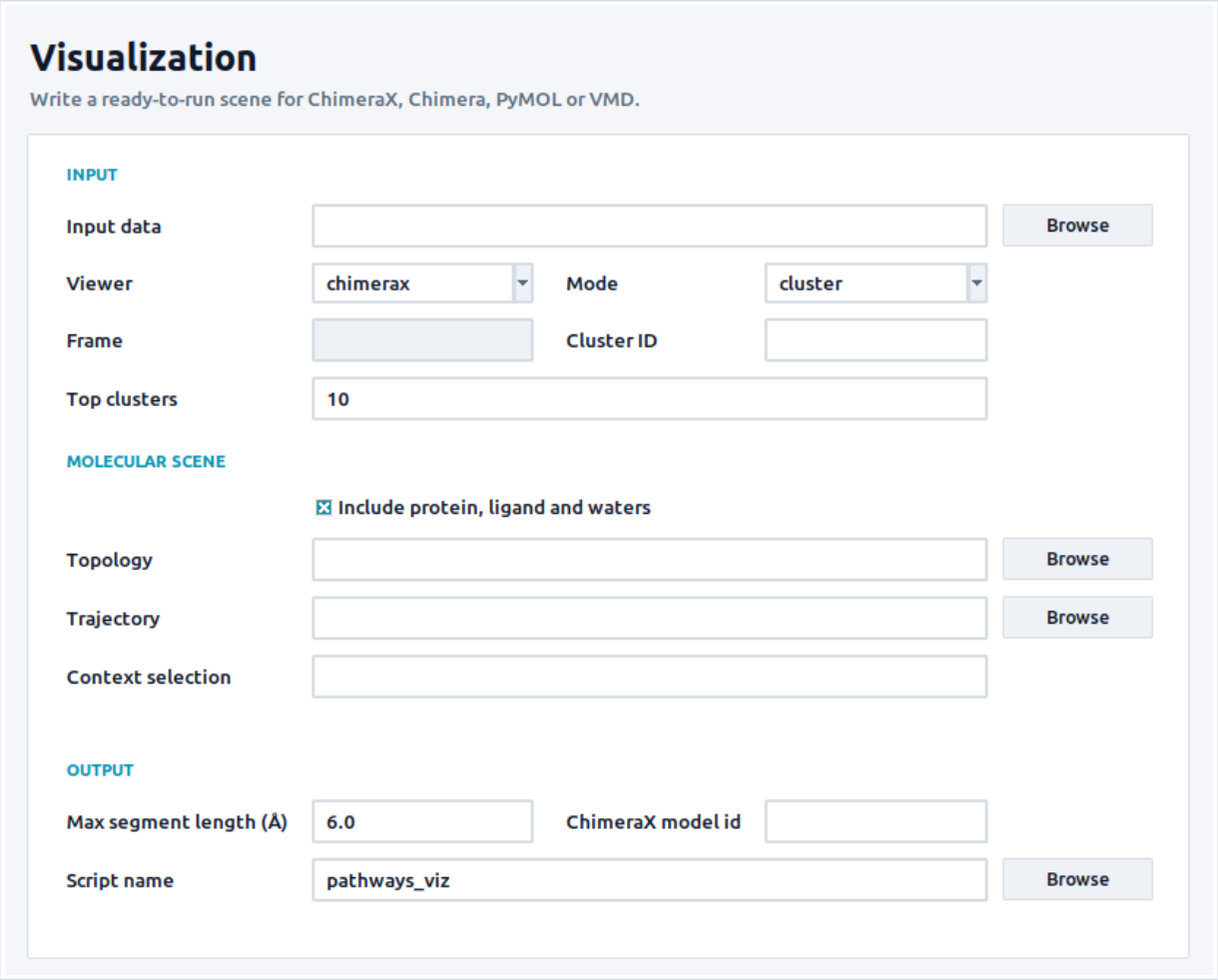
Click **Run clustering**. When it finishes, a confirmation window reports how many channels were identified and lists the saved file. **Next: Visualization** opens step 3 with the cluster file already selected.




*The confirmation shown when clustering finishes.*

# 6. Step 3 - Visualization

Writes a script that opens a complete scene in your molecular viewer: the protein as a cartoon, the ligand and the bridging waters as sticks, and the water network drawn on top.




*Step 3, Visualization: viewer, mode and the molecular-scene section.*

## Choosing what to draw

| Mode | Input file | What the scene shows |
|---|---|---|
| **cluster** | cluster file (step 2) | The protein and ligand with the medoid (most representative) path of each cluster as a colour-coded tube, and a legend with each cluster's occupancy. **Top clusters** limits the picture to the most occupied ones. Enter a **Cluster ID** to rebuild that one cluster as real water molecules at the frame it comes from. |
| **frame** | results file (step 1) | One frame: the real water molecules (oxygen and hydrogens) and every water bridge as a dashed bond. Choose the frame number in **Frame**. |
| **density** | results file (step 1) | Every path of every frame as thin transparent tubes, showing where water bridges form most often. |

## Molecular scene

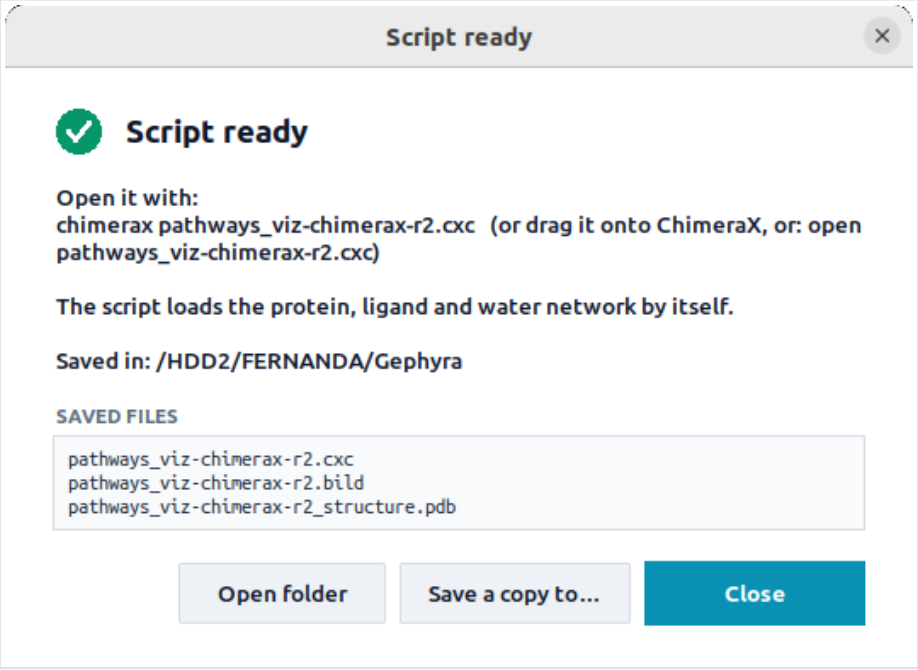
With **Include protein, ligand and waters** ticked (the default), Gephyra re-opens the analysed trajectory and writes a small structure file (`<name>_structure.pdb`) next to the script, which the script loads by

itself. The topology and trajectory are read from the data file; the line under the section says which ones. Fill in **Topology** and **Trajectory** only for older result files that do not record them, or if the files have moved.

Untick it to write only the network geometry, then open the analysed structure in the viewer yourself before running the script.

## Writing and opening the script

Pick the **Viewer**, click **Write script**, and the confirmation shows the exact command that opens it. Do not open the structure first: the script does it.

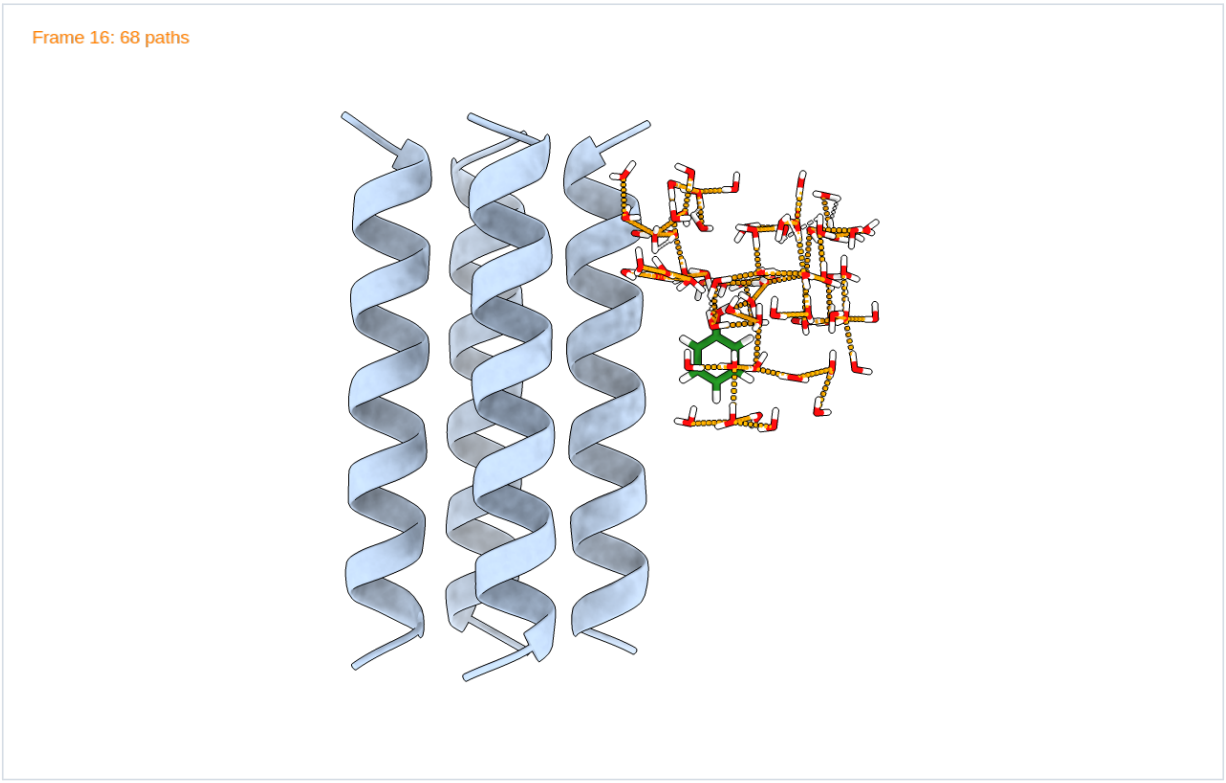



*The 'Script ready' confirmation with the command that opens the script.*

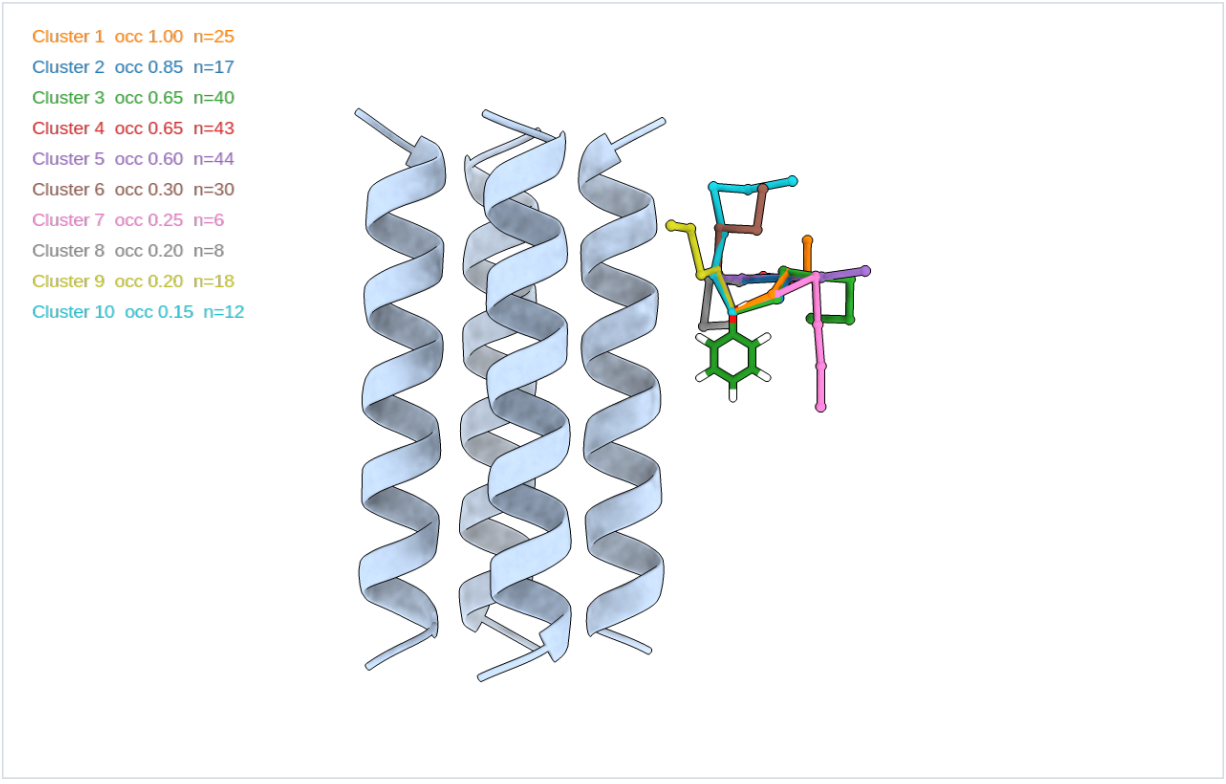
| Viewer | Script | How to open it |
|---|---|---|
| **UCSF ChimeraX** | .cxc | `chimerax scene.cxc`, drag the file onto the window, or type `open scene.cxc`. |
| **UCSF Chimera** | .py | `chimera scene.py`, or File > Open... |
| **PyMOL** | .py | `pymol scene.py`, or `run scene.py` inside PyMOL. |
| **VMD** | .tcl | `vmd -e scene.tcl`, or `source scene.tcl` in the Tk Console. The `-e` matters: without it VMD tries to read the script as a molecule. |

**VMD scripts must be run with `vmd -e scene.tcl`.** Without `-e`, VMD treats the script as a molecule file, reports *Unable to determine file type*, and draws nothing. Inside a running VMD, use `source scene.tcl` in Extensions > Tk Console instead.

Running a script again replaces the previous Gephyra scene.




*A frame scene in ChimeraX: protein cartoon, ligand, water molecules and dashed water bridges (hypothetical example system).*




*A cluster scene: colour-coded medoid paths around the protein, with the legend (hypothetical example system).*

# 7. Output files

| File | Written by | Contents |
|---|---|---|
| **results.jsonl** | Step 1 | One line per analysed frame with every water bridge (atoms, coordinates, H-quality score), plus the analysed topology/trajectory and settings. |
| **summary.csv** | Step 1 (optional) | The same paths as a table. |
| **clustered_pathways.json** | Step 2 | The channels: size, occupancy, persistence, H-quality and the medoid path of each, plus the analysed system. |
| **name.cxc / .py / .tcl** | Step 3 | The viewer script. |
| **name_structure.pdb** | Step 3 | The protein, ligand and bridging waters loaded by the script. |
| **name.bild, name.pb** | Step 3 | Network geometry and pseudobonds used by the Chimera and ChimeraX scripts. |

H-quality (Z) sums the geometric quality of every route through a channel. It is not a probability and can exceed 1 for channels with many parallel routes; *hquality_norm* = Z/(1+Z) is a bounded version for comparisons. See the README for details.

# 8. Troubleshooting

| Message or symptom | What to do |
|---|---|
| **Text looks blocky or pixelated** | Install the anti-aliased Tk build: `conda install -c conda-forge "tk=*=xft*"` (section 2). |
| **Root selection returned no atoms** | Check the residue and atom names in your topology; the selection uses MDAnalysis syntax. |
| **A trajectory file is required** | Choose a trajectory. For a single structure, choose a .pdb as the trajectory. |
| **Mode 'cluster' needs the cluster file** | In step 3, cluster mode reads the file from step 2; frame and density modes read the results file from step 1. |
| **The data file does not record its structure** | The file comes from an older version of Gephyra. Fill in Topology and Trajectory in the Molecular scene section. |
| **VMD: Unable to determine file type** | Open the script with `vmd -e scene.tcl`. |
| **Too many clusters to read the picture** | Lower **Top clusters**, or draw a single cluster with **Cluster ID**. |
| **Bridges or protein cut at the box edge** | Image/centre the trajectory before the analysis (for example cpptraj autoimage or gmx trjconv -pbc mol -center). |
| **Something else went wrong** | Start the interface with `gephyra -v gui` to see full error details in the terminal, and include them when reporting the problem. |

# 9. Command-line equivalents

Every run writes its command-line equivalent to the log, and **Copy CLI command** copies it, so a setup made in the interface can be repeated in scripts or on a cluster. For example:

```
gephyra calculate --topo system.prmtop --traj run.dcd \
    --root "resname LIG and name O4" --stride 20 --output results.jsonl
gephyra cluster --data results.jsonl --output clustered_pathways.json
gephyra visualize --data clustered_pathways.json --format chimerax \
    --mode cluster --top_clusters 10 --output cluster_scene
```

# 10. Video walkthrough

A short video walkthrough of the interface will be linked here.